Exploratory Data Analysis

- DATA CLEANING
- DATA PRE-PROCESSING
- DATA INSIGHTS
- VISUALIZATION
- REPORT



Goals of Exploratory Data Analysis

1. Understand the Data

- **Goal:** Get a comprehensive understanding of the dataset.
  - Assess the structure, size, and types of variables.
  - Identify the nature of the problem (e.g., regression, classification, clustering).

---

2. Identify Data Quality Issues

- **Goal:** Detect and handle issues that might negatively impact model performance.
  - **Missing values:** Decide whether to impute, drop, or ignore them.
  - **Outliers:** Identify and decide how to handle extreme values.
  - **Duplicate data:** Remove redundant records.

3. Understand Feature Distributions

- **Goal:** Analyze the distribution of each feature to identify patterns and anomalies.
    - Visualize **numerical features** using histograms or boxplots.
    - Check the balance of **categorical features** (e.g., class imbalance in classification problems).

---

4. Identify Relationships Between Variables

- **Goal:** Understand dependencies between features and their impact on the target variable.
    - **Correlation analysis**: Identify strong positive or negative correlations.
    - **Feature interactions**: Use scatter plots or pair plots to study relationships.

5. Reduce Dimensionality

- **Goal:** Identify irrelevant or redundant features to simplify the dataset.
    - Drop features with little variance or high multicollinearity.
    - Use techniques like PCA or feature selection methods if necessary.

---

6. Identify Key Predictors

- **Goal:** Determine which features are most important for predicting the target variable.
    - Perform univariate analysis of the target variable against potential predictors.
    - Use statistical tests or feature importance techniques to rank features.



7. Handle Class Imbalance (for Classification Problems)

- **Goal:** Address imbalances in target classes to avoid biased model predictions.
    - Techniques: Oversampling (e.g., SMOTE) or undersampling.

---

8. Guide Feature Engineering

- **Goal:** Identify transformations or new features that might improve model performance.
    - Log-transform skewed features.
    - Create interaction terms or combine features where relationships exist.



9. Detect Patterns and Anomalies

- **Goal:** Uncover hidden insights that might influence the model or business understanding.
    - Look for trends, seasonality, or other patterns in time-series data.
    - Detect rare but important events (e.g., fraud detection).

---

10. Prepare Data for Modeling

- **Goal:** Transform the dataset into a form suitable for machine learning algorithms.
    - Encode categorical variables (e.g., one-hot encoding, label encoding).
    - Scale numerical features (e.g., standardization, normalization).
    - Split data into training, validation, and testing sets.



Types of Exploratory Data Analysis

1. Univariate Analysis

- **Definition:** Analysis of a single variable to understand its distribution, central tendency, and spread.
- **Goal:** Summarize and describe the characteristics of one variable.
- **Techniques/Plots:**
  - Numerical Data: Histograms, Boxplots, Density plots, Summary statistics (mean, median, mode, variance, standard deviation).
  - Categorical Data: Bar plots, Pie charts, Frequency tables.
- **Example:**
  - Variable: Student Marks
  - Insights: What is the average score? Are there any outliers?

2. Bivariate Analysis

- **Definition:** Analysis of the relationship between two variables.
- **Goal:** Understand how one variable impacts or correlates with another.
- **Techniques/Plots:**
  - **Numerical vs. Numerical:** Scatterplots, Correlation coefficient, Line plots.
  - **Numerical vs. Categorical:** Boxplots, Violin plots, Grouped bar charts.
  - **Categorical vs. Categorical:** Contingency tables, Stacked bar charts, Heatmaps.
- **Example:**
  - Variables: Marks vs. Attendance
  - Insights: Does higher attendance lead to better marks?

3. Multivariate Analysis

- **Definition:** Analysis of three or more variables simultaneously to understand complex relationships.
- **Goal:** Explore interactions among multiple variables and their combined effect.
- **Techniques/Plots:**
  - Pairplot (e.g., Seaborn pairplot).
  - Heatmap for correlation among multiple numerical variables.
  - 3D Scatterplots (if three variables).
  - Dimensionality reduction techniques (e.g., PCA, t-SNE).
- **Example:**
  - Variables: Marks, Attendance, Gender
  - Insights: Do attendance and gender together influence marks?

4. Time Series Analysis

- **Definition:** Analysis of data points collected or recorded at specific time intervals.
- **Goal:** Identify trends, seasonal patterns, and cyclic behavior over time.
- **Techniques/Plots:**
  - Line plots for time-series trends.
  - Seasonal decomposition of time series (STL decomposition).
  - Moving averages and smoothing.
  - Autocorrelation and partial autocorrelation plots.
- **Example:**
  - Variable: Monthly sales data
  - Insights: Are there seasonal peaks in sales during holidays?

Type	Variables	Goal	Example Question
Univariate	1 variable	Understand its distribution or summary.	What is the average score?
Bivariate	2 variables	Understand relationships or dependencies.	Does attendance influence marks?
Multivariate	3 or more variables	Explore interactions among variables.	How do attendance and gender together affect marks?
Time Series	1 variable (over time)	Analyze trends and patterns over time.	How do sales vary throughout the year?

Insights About a Dataset

1. Load the Dataset

- Use Pandas to load datasets from CSV, Excel, or other formats.

In [5]:
```
import pandas as pd

# Load a sample dataset (Titanic dataset)
url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
data = pd.read_csv(url)
data
```

Out[5]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
...	...	...	...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.0000	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.7500	NaN	Q

891 rows × 12 columns

2. View the Dataset Structure

- `data.head()` : Displays the first few rows.
- `data.tail()` : Displays the last few rows.

In [8]:
```
# First 2 rows
print(data.head(2))

# Last 2 rows
print(data.tail(2))
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1

   Parch     Ticket     Fare Cabin Embarked
0      0  A/5 21171   7.2500   NaN        S
1      0   PC 17599  71.2833   C85        C
     PassengerId  Survived  Pclass                     Name   Sex   Age  SibSp  \
889          890         1       1  Behr, Mr. Karl Howell  male  26.0      0
890          891         0       3    Dooley, Mr. Patrick  male  32.0      0

     Parch  Ticket   Fare Cabin Embarked
889      0  111369  30.00  C148        C
890      0  370376   7.75   NaN        Q
```

3. Get Basic Information

- `data.info()` : Shows column names, data types, and non-null counts.
- `data.shape` : Gives the dimensions (rows, columns).
- `data.columns` : Lists column names.
- `data.dtypes` : Shows data types of each column.

In [10]:
```
# Dataset information
print(data.info())
# Dataset dimensions
print("Shape of the dataset:", data.shape)
# Column names
print("Columns:", data.columns)
# Data types
print("Data types:\n", data.dtypes)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
Shape of the dataset: (891, 12)
Columns: Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
Data types:
 PassengerId      int64
Survived         int64
Pclass           int64
```

4. Statistical Summary

- `data.describe()` : Generates summary statistics for numerical columns.
- `data.describe(include='all')` : Includes categorical columns.
- `data.mean()`, `data.median()`, `data.std()` : Individual stats.

In [11]:
```
# Summary statistics for numerical columns
print(data.describe())

# Summary statistics for all columns (including categorical)
print(data.describe(include='all'))

# Calculate mean and median for a specific column (e.g., Age)
print("Mean Age:", data['Age'].mean())
print("Median Age:", data['Age'].median())
```

```
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
       PassengerId    Survived      Pclass                      Name    Sex  \
count   891.000000  891.000000  891.000000                       891    891
unique         NaN         NaN         NaN                       891      2
top            NaN         NaN         NaN  Braund, Mr. Owen Harris   male
freq           NaN         NaN         NaN                         1    577
mean    446.000000    0.383838    2.308642                       NaN    NaN
```

5. Check for Missing Data

- `data.isnull().sum()` : Count missing values for each column.
- `data.isnull().any()` : Check if a column has any missing values.

In [12]:
```
# Count missing values in each column
print("Missing values:\n", data.isnull().sum())

# Check if any column has missing data
print("Any missing data?", data.isnull().any().any())
```

```
Missing values:
 PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
Any missing data? True
```

6. Check for Unique and Duplicate Values

- `data.duplicated().sum()` : Count duplicate rows.
- `data['column_name'].unique()` : List unique values in a column.
- `data['column_name'].nunique()` : Count unique values in a column.

In [13]:
```
# Count duplicate rows
print("Number of duplicate rows:", data.duplicated().sum())

# list Unique values in the 'Sex' column
print("Unique values in 'Sex':", data['Sex'].unique())

# Count of unique values in the 'Pclass' column
print("Number of unique Pclass:", data['Pclass'].nunique())
```

```
Number of duplicate rows: 0
Unique values in 'Sex': ['male' 'female']
Number of unique Pclass: 3
```

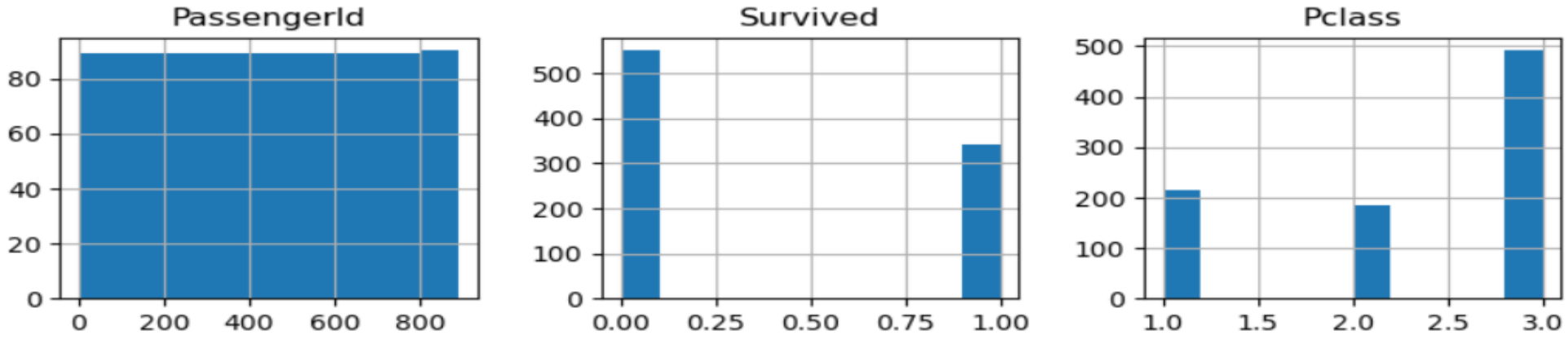
7. Check Value Distribution

- `data['column_name'].value_counts()` : Frequency distribution of categorical columns.
- `data.hist()` : Histogram for numerical columns.

In [14]:
```
# Frequency of values in the 'Embarked' column
print(data['Embarked'].value_counts())
# Plot histograms for all numerical columns
data.hist(figsize=(10, 8))
```

```
S    644
C    168
Q     77
Name: Embarked, dtype: int64
```

Out[14]:
```
array([[<Axes: title={'center': 'PassengerId'}>,
        <Axes: title={'center': 'Survived'}>,
        <Axes: title={'center': 'Pclass'}>],
       [<Axes: title={'center': 'Age'}>,
        <Axes: title={'center': 'SibSp'}>,
        <Axes: title={'center': 'Parch'}>],
       [<Axes: title={'center': 'Fare'}>, <Axes: >, <Axes: >]],
      dtype=object)
```



Aspect	Key Function	Purpose	Example Output
Dataset size	`data.shape`	Size of the dataset (rows, columns).	`(891, 12)`
Column names and data types	`data.info()`	Overview of column names and types.	`Age: float, Sex: object`
Preview dataset	`data.head()`	Displays the first few rows of data.	First 5 rows
Summary statistics	`data.describe()`	Provides mean, median, and other metrics.	Mean Age: 29.7, Max Fare: 512
Missing values	`data.isnull().sum()`	Identifies missing data in each column.	`Age: 177 missing`
Unique values	`data['Embarked'].unique()`	Categorical data distribution.	`['S', 'C', 'Q']`

Handling Missing Data

2.1 Remove Rows or Columns

- Use this method if the data is minimal or irrelevant.

Commands:

python

Copy code

```python
# Drop rows with missing values
data_dropped_rows = data.dropna()

# Drop specific columns with missing values
data_dropped_columns = data.drop(columns=['Cabin'])
```

2.2 Fill Missing Values (Imputation)

a. Fill with a Default Value

- Replace missing values with a fixed number (e.g., `0`) or string (e.g., `'Unknown'`).

Commands:

python

Copy code

```python
# Fill missing 'Age' with 0
data['Age'].fillna(0, inplace=True)

# Fill missing 'Cabin' with 'Unknown'
data['Cabin'].fillna('Unknown', inplace=True)
```

b. Fill with Statistical Values

- Use **mean**, **median**, or **mode** for numerical columns and **mode** for categorical columns.

Commands:

python

Copy code

```python
# Fill missing 'Age' with mean
data['Age'].fillna(data['Age'].mean(), inplace=True)

# Fill missing 'Embarked' with mode
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

c. Forward or Backward Fill

- Use previous or next values in the dataset.

Commands:

python

Copy code

```python
# Forward fill
data['Age'].fillna(method='ffill', inplace=True)

# Backward fill
data['Age'].fillna(method='bfill', inplace=True)
```

2.3 Interpolation

- For numerical data, interpolate values between existing data points.

Commands:

python

Copy code

```python
# Linear interpolation
data['Age'] = data['Age'].interpolate(method='linear')
```

2.4 Advanced Techniques

- Use machine learning models to predict missing values based on other features.

Example Using `KNNImputer`:

python

Copy code

```python
from sklearn.impute import KNNImputer

# Create KNNImputer object
imputer = KNNImputer(n_neighbors=5)

# Apply imputation
data[['Age', 'Fare']] = imputer.fit_transform(data[['Age', 'Fare']])
```



When to Use Each Method

Method	When to Use
Drop Rows/Columns	When the column/row has too many missing values or is not critical for analysis.
Fill with Defaults	When a missing value represents an absence (e.g., no cabin = 'Unknown').
Statistical Impute	For numerical columns where the mean or median is meaningful.
Forward/Backward Fill	When data is time-series or sequential.
Interpolation	When values follow a trend or pattern.
ML-Based Impute	When missing values depend on other features.



Data Visualization Tools: Graphs, Maps, and Plots

Data visualization tools and techniques help represent data visually, making patterns, trends, and insights easier to understand. Here are the most commonly used tools and their applications in graphs, maps, and plots.

Common Data Visualization Libraries and Tools

- **Matplotlib:** Versatile for creating static, interactive, and animated visualizations in Python.
- **Seaborn:** Built on Matplotlib, it simplifies creating attractive and informative statistical graphics.
- **Plotly:** Interactive, browser-based visualizations with support for graphs, maps, and dashboards.
- **Tableau/Power BI:** Industry-standard tools for building interactive dashboards and charts.
- **GeoPandas/Kepler.gl:** Tools specifically designed for geospatial data and map visualizations.
- **D3.js:** A JavaScript library for producing sophisticated interactive visualizations.

1. Graphs

Graphs are visual representations of data, used to show relationships between variables, trends, and comparisons. Common types include line graphs, bar graphs, and scatter plots.

Definition: A graph is a diagram that represents data points and relationships between variables using various shapes like lines, bars, dots, or areas.

Uses:

- To visualize trends or changes over time (e.g., line graphs for time-series data).
- To compare values across different categories (e.g., bar graphs).
- To show relationships between two or more variables (e.g., scatter plots).

Advantages:

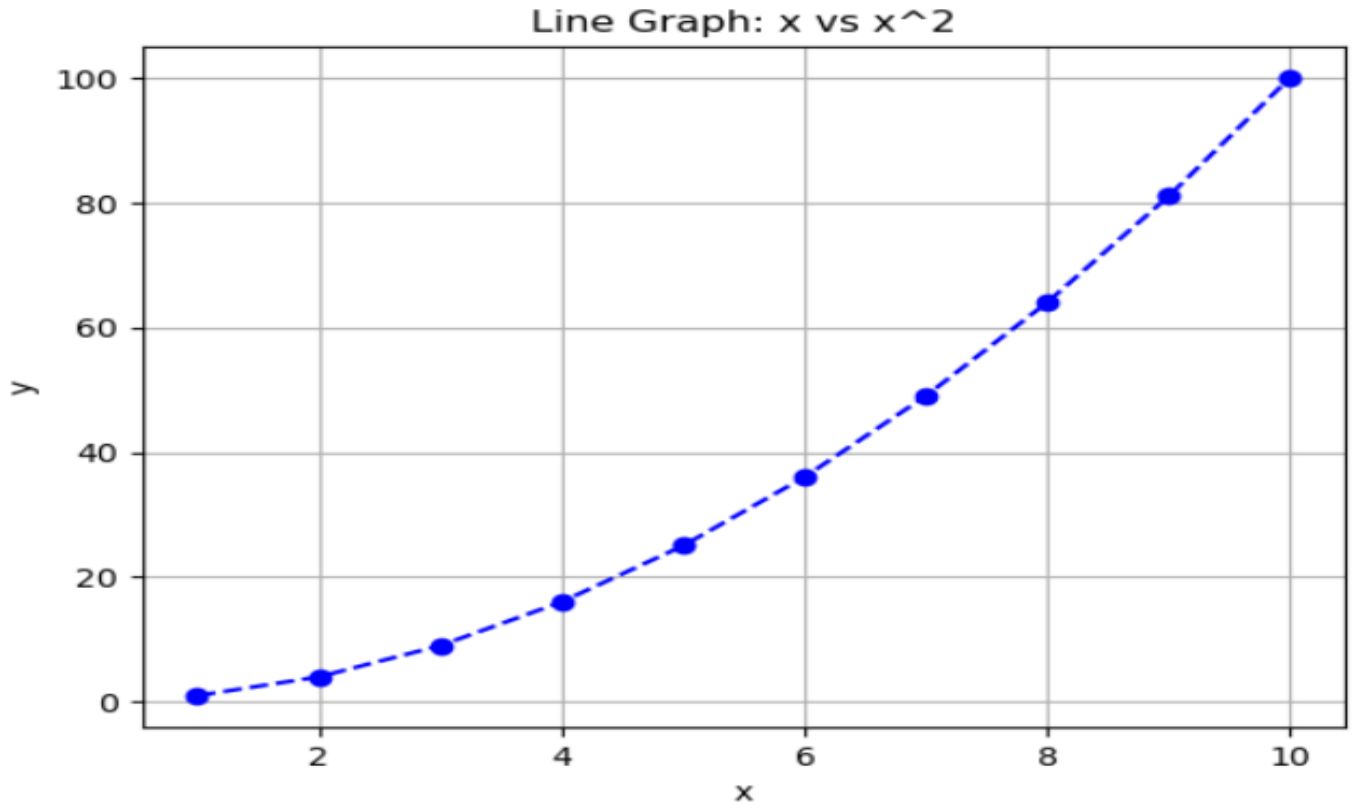
1. **Simplifies Complex Data**: Graphs make complex data easier to interpret by summarizing it visually.
2. **Helps with Comparisons**: They allow easy comparisons between different categories or groups.
3. **Highlight Trends**: Great for visualizing trends and patterns over time or across variables.

Disadvantages:

1. **Misleading Representation**: Poorly constructed graphs can mislead the viewer (e.g., distorted axes).
2. **Over-simplification**: Important details might be overlooked or missed in the simplification process.
3. **Context Dependency**: The interpretation of a graph depends on the context and prior knowledge.

1.Line Graph: Used to display trends over time.

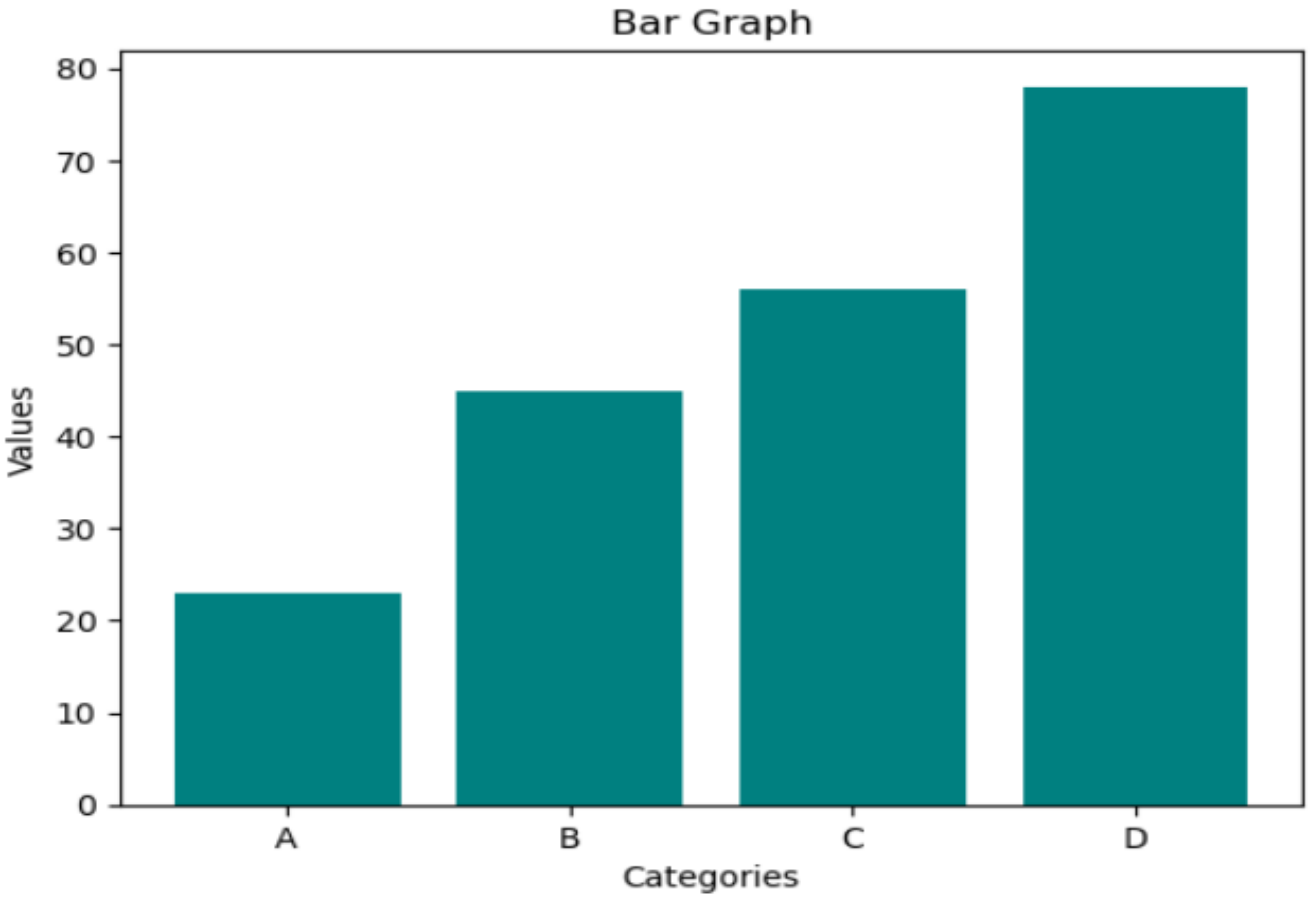
[15]:
```
import matplotlib.pyplot as plt
import numpy as np
x = np.arange(1, 11)
y = x**2
plt.plot(x, y, marker='o', linestyle='--', color='b')
plt.title("Line Graph: x vs x^2")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.show()
```



Bar Graph: Compares categorical data.

In [16]:
```
categories = ['A', 'B', 'C', 'D']
values = [23, 45, 56, 78]

plt.bar(categories, values, color='teal')
plt.title("Bar Graph")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.show()
```

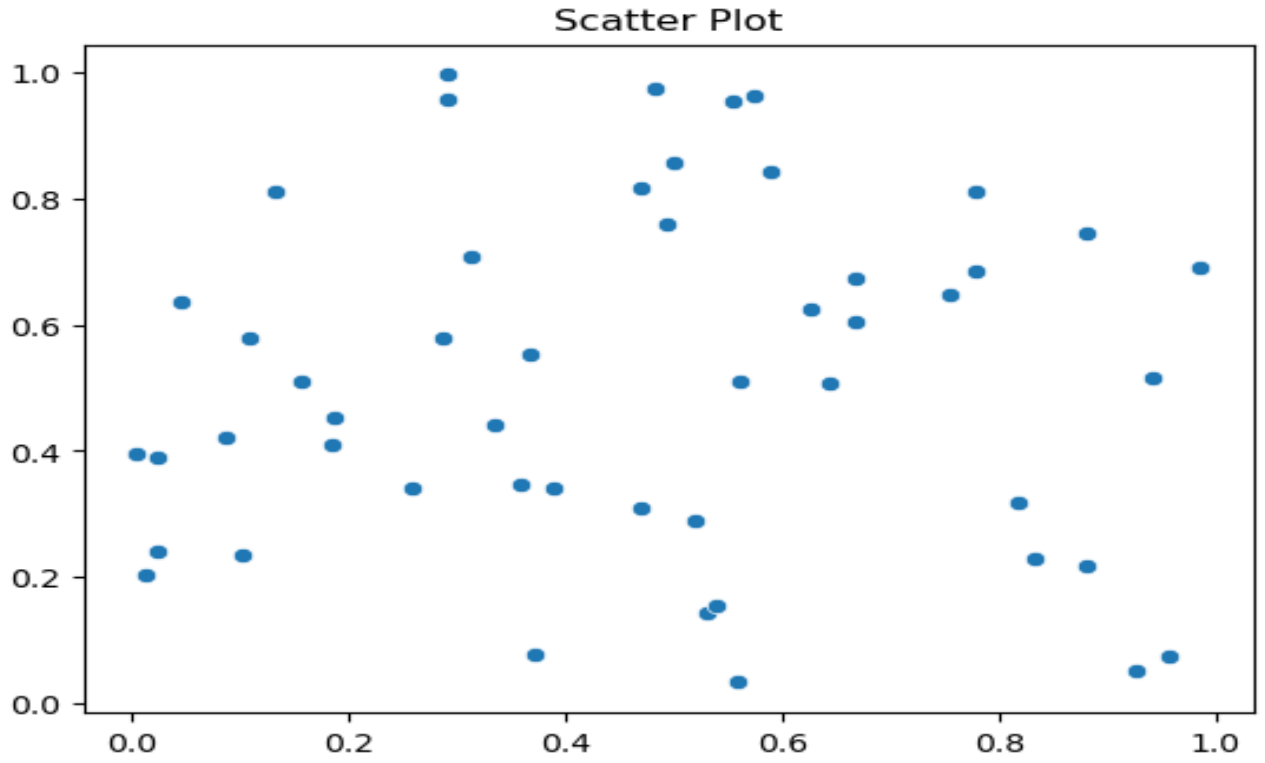


Scatter Plot: Shows relationships between two numerical variables.

In [17]:
```
import seaborn as sns

# Random data
x = np.random.rand(50)
y = np.random.rand(50)

sns.scatterplot(x=x, y=y)
plt.title("Scatter Plot")
plt.show()
```



2. Maps

Maps are a type of data visualization that displays spatial or geographical data. They are particularly useful when dealing with location-based datasets, showing geographical relationships, trends, and patterns.

Definition: A map is a visual representation of geographical data or location-based information, often displaying points, boundaries, and geographic attributes.

Uses:

- **Geospatial Data:** Maps are used to display locations, such as cities, countries, or continents.
- **Geographical Analysis:** They help analyze trends, like population distribution, weather patterns, or sales by location.
- **Navigation:** Used in routing and navigation (e.g., Google Maps).

Advantages:

1. **Visualizes Geographical Relationships**: Helps in understanding spatial distributions and connections.
2. **Clear Location Representation**: Makes it easy to interpret location-based data.
3. **Interactive Exploration**: Tools like Folium or Google Maps allow for interactive zooming and panning.

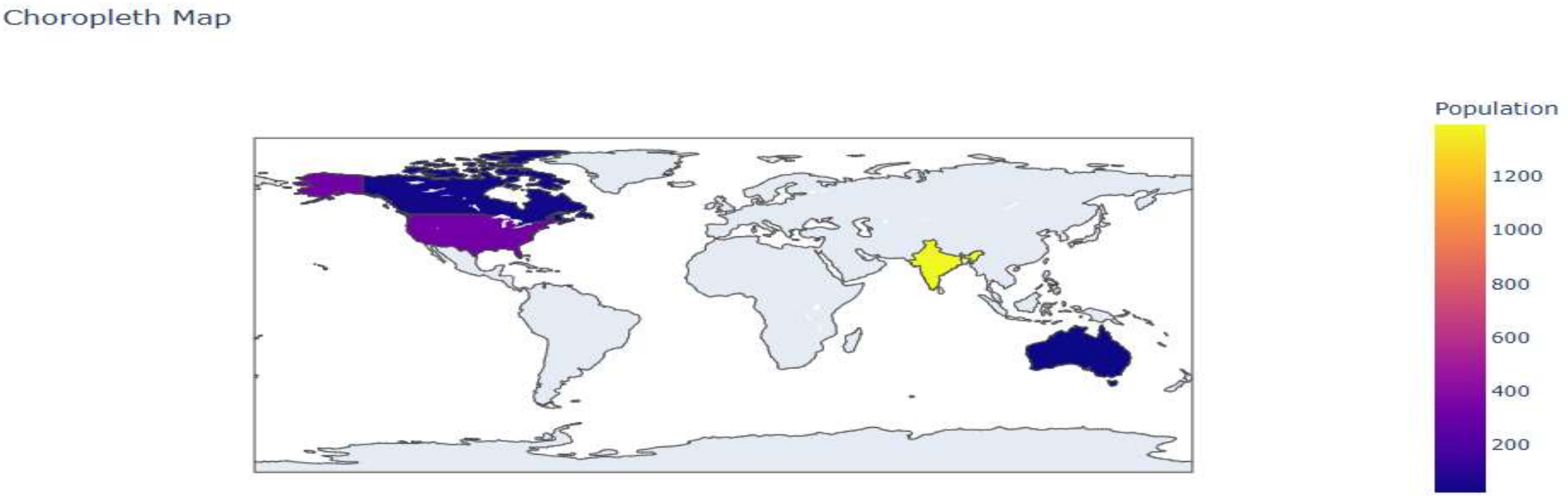
Disadvantages:

1. **Requires Geospatial Data**: Maps are not useful without geographic data or coordinates.
2. **Complexity with Large Datasets**: Handling large amounts of geospatial data on a map can be cumbersome.
3. **Resolution Limitations**: Maps can become unclear or too crowded when displaying too many data points.



1.Choropleth Map (Plotly):

In [18]:
```
import plotly.express as px
# Example data
data = {'Country': ['USA', 'Canada', 'India', 'Australia'],
        'Population': [331, 38, 1393, 25]}
df = pd.DataFrame(data)
fig = px.choropleth(df,
                    locations="Country",
                    locationmode='country names',
                    color="Population",
                    title="Choropleth Map")
fig.show()
```



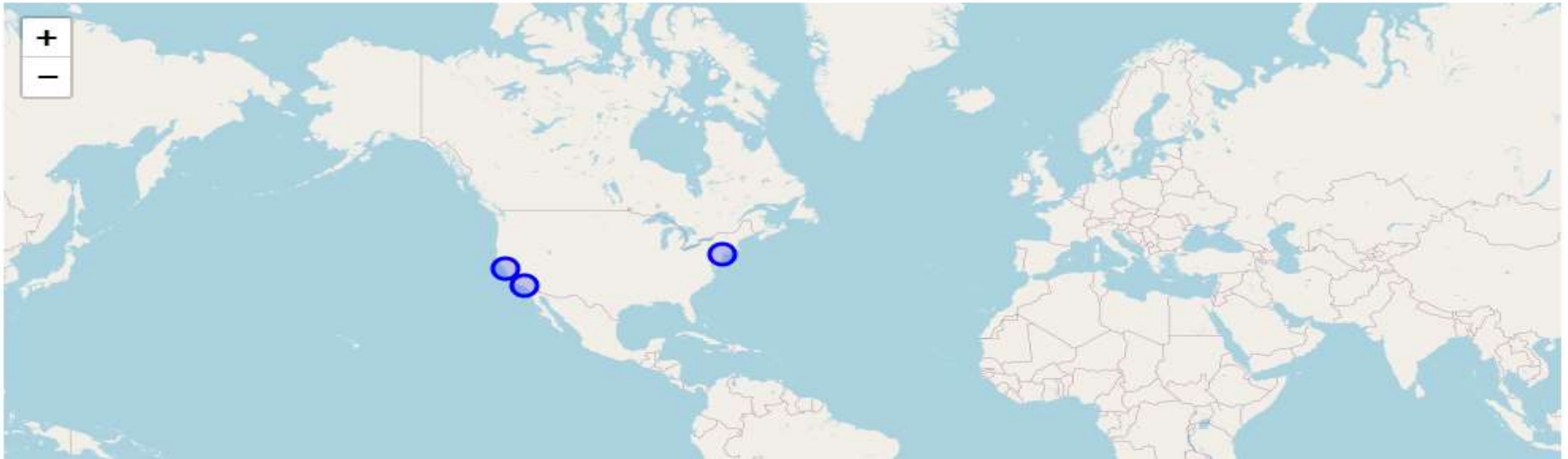
Scatter Map Example Using Folium

[25]:
```
import folium
# Create a map centered at a specific location (latitude, longitude)
map_center = [37.7749, -122.4194]  # San Francisco coordinates
m = folium.Map(location=map_center, zoom_start=12)

# Add markers at specific coordinates (scatter plot effect)
locations = [
    [37.7749, -122.4194],  # San Francisco
    [34.0522, -118.2437],  # Los Angeles
    [40.7128, -74.0060],   # New York City
]
# Add a marker for each location
for loc in locations:
    folium.CircleMarker(location=loc, radius=8, color='blue', fill=True, fill_color='blue').add_to(m)

# Display the map directly in Jupyter Notebook
m
```

t[25]:



3. Plots

Plots are graphical representations of data, primarily used to depict the relationships or distributions of data points. Examples include histograms, boxplots, and heatmaps.

Definition: A plot is a visual representation of data on a coordinate plane, often to show relationships, distributions, or trends.

Uses:

- **Statistical Analysis:** Plots like histograms and boxplots help visualize distributions and outliers.
- **Comparison:** Plots help compare multiple variables or datasets.
- **Feature Relationships:** Scatter plots, line plots, and correlation plots help understand relationships between variables.

Advantages:

1. **Statistical Insights:** Useful for statistical analysis like distributions and correlations.
2. **Clear Distribution Representation:** Provides insights into data distribution, such as normal or skewed distributions.
3. **Easy to Compare:** Plots like scatter plots allow easy comparison of different datasets.

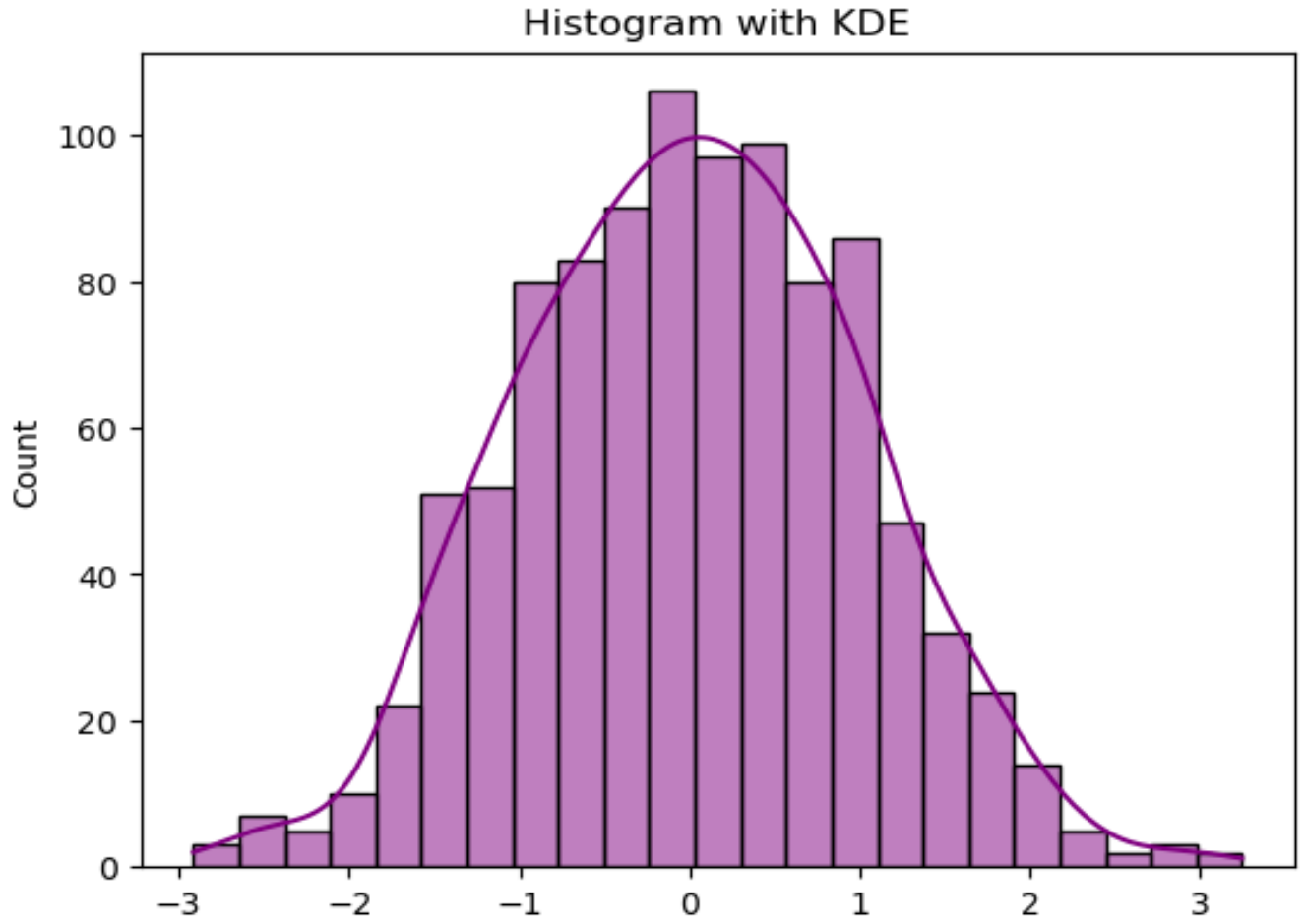
Disadvantages:

1. **Can Overwhelm:** With complex datasets, plots like scatter plots or heatmaps can become cluttered and difficult to read.
2. **Requires Clean Data:** The plot's effectiveness heavily relies on clean, well-structured data.
3. **Difficult for Large Datasets:** When dealing with large datasets, plots can become noisy or difficult to interpret without proper aggregation.



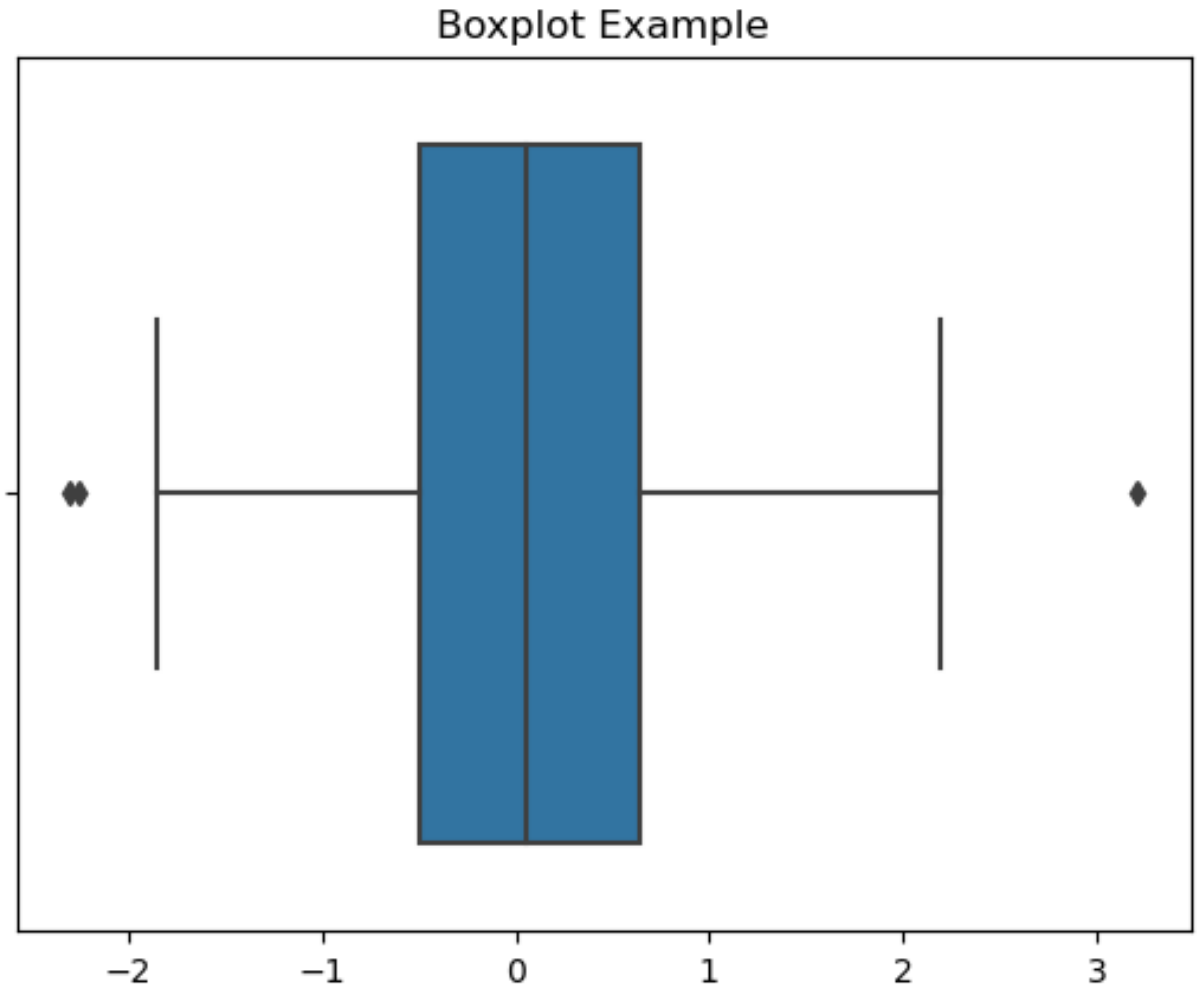
1.Histogram (Distribution Plot):

In [19]:
```
sns.histplot(data=np.random.randn(1000), kde=True, color='purple')
plt.title("Histogram with KDE")
plt.show()
```



Boxplot (Outlier Detection):

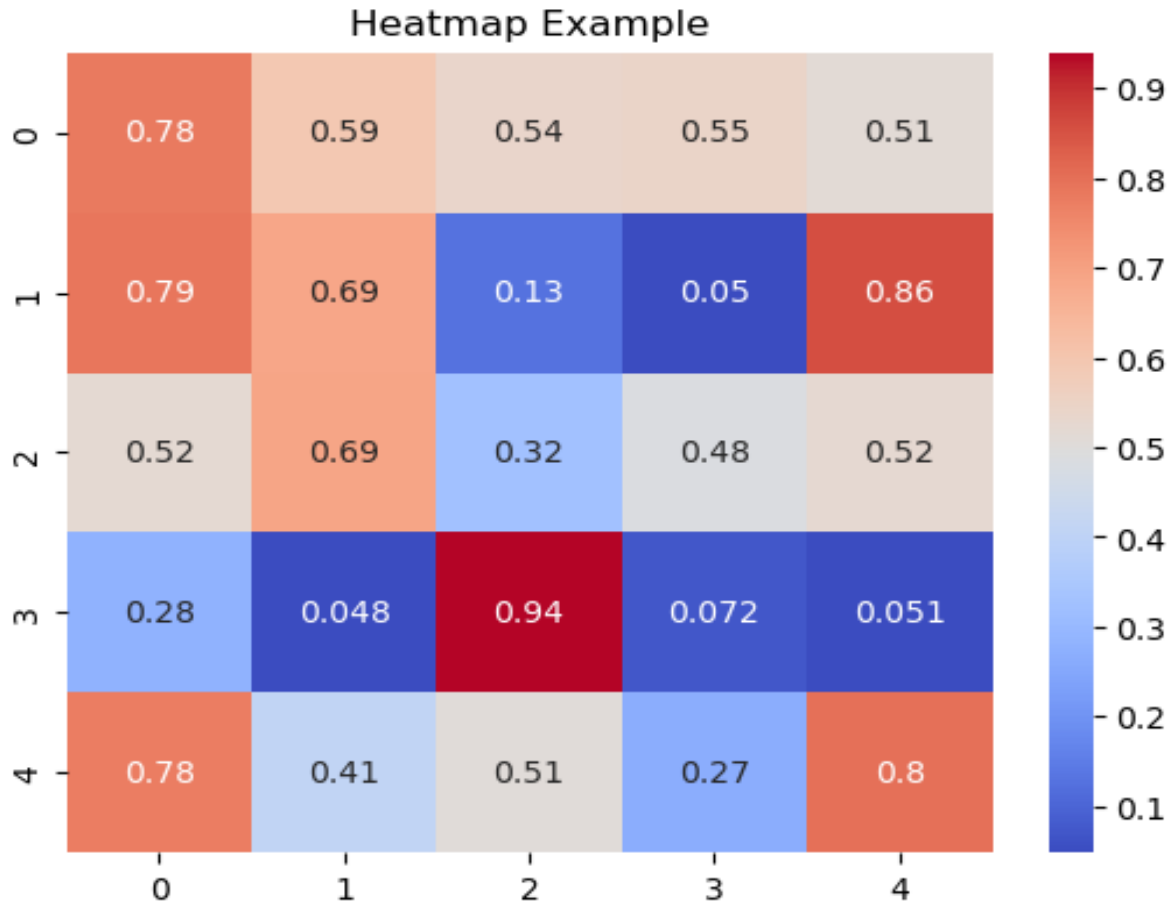
In [20]:
```
sns.boxplot(x=np.random.randn(100))
plt.title("Boxplot Example")
plt.show()
```



Heatmap (Correlation or Matrix Data):

In [21]:
```
import numpy as np

# Random correlation matrix
data = np.random.rand(5, 5)
sns.heatmap(data, annot=True, cmap='coolwarm')
plt.title("Heatmap Example")
plt.show()
```



Type	Definition	Uses	Advantages	Disadvantages
Graphs	Visual representation of data relationships	Visualizing trends, comparisons, relationships	Simplifies data, highlights trends, easy comparisons	Misleading representation, over-simplification, context-dependent
Maps	Geographical visualization of data	Geospatial analysis, geographical distribution	Visualizes spatial data, clear location representation	Requires geospatial data, complex for large datasets
Plots	Visual representation of data on a coordinate plane	Statistical analysis, feature relationships, comparisons	Statistical insights, clear distribution representation	Can overwhelm with complexity, requires clean data

Type	Best Used For
Line Graph	Time-series data or trends.
Bar Graph	Comparison of categories.
Scatter Plot	Relationships between two numerical variables.
Heatmap	Correlation or matrix data visualization.
Choropleth Map	Geographical distribution of a numerical variable.
Boxplot	Identifying outliers and distribution spread.
Histogram	Understanding distributions and frequency of numerical data.


Thank you